

Assignment 2:

Capture-the-Flag (CTF)

Name: Rishabh Ray
Student ID: 34525416

Hybrid Symbolic–Heuristic Multi-Agent Planning for Capture-the-Flag

Abstract

This report describes the design and development of my two-agent Capture-the-Flag (CTF) system for **FIT5222 – Planning and Automated Reasoning**.

The project brings together ideas taught throughout the unit — from early topics on **search and path planning** to later ones on **PDDL reasoning, uncertainty, and explainable planning**.

My final design combines a **PDDL-based high-level planner** with a **risk-aware A*** low-level planner and a **particle-filter belief tracker** for handling partial observability. The two agents cooperate through a shared blackboard.

I also experimented with more advanced ideas, such as Monte Carlo Tree Search, TD-Lambda learning, and opponent modelling, before settling on this hybrid symbolic-heuristic approach.

Overall, the final system performs well in the contest environment while remaining fast, interpretable, and closely aligned with the core principles covered in FIT5222.

1. Introduction

The goal of this assignment was to design intelligent agents that could reason, plan, and act in a competitive Pac-Man-style Capture-the-Flag game.

This task follows: Weeks 1–6 focused on search and path planning, and Weeks 7–12 explored AI planning, uncertainty, and explainability.

My agents follow the rational-agent model from Week 1: they **perceive**, **reason**, and **act** to maximise expected score.

I split this reasoning into four layers :

Layer	Related FIT5222 Topic	Function
Strategic	Weeks 7–9 – PDDL & Planning as Heuristic Search	Pick a high-level intent
Tactical	Weeks 2–4 – Heuristic Path Planning	Find safe paths with A*
Perceptual	Week 10 – Uncertainty & MDPs	Track hidden enemies
Cooperative	Week 5 – Multi-Agent Path Finding	Coordinate with teammate

2. Problem Context

The Pacman CTF domain is a great test-bed for everything in this unit.

It's **multi-agent**, **dynamic**, **partially observable**, and **sequential**, the same kinds of environments we analysed in the “Problem Model” lecture (Week 8).

Each agent must plan under uncertainty, switch between offensive and defensive modes, and cooperate with its teammate.

This means single-agent search (Weeks 1–4) is not enough; we also need multi-agent reasoning (Week 5) and symbolic planning (Weeks 7–9).

3. System Overview

Each turn, my agent goes through the same perception-reasoning-action loop:

1. Update enemy beliefs (particle filters).
2. Extract predicates to describe the world in PDDL form.
3. Use the **PDDLPlanner** to choose a high-level action.
4. Compute an A* path to the target of that action.
5. Take the first move and update shared data on the blackboard.

Both agents run this loop independently but cooperate through shared memory.

4. PDDL-Based Strategic Planning

4.1 Domain Overview

Following the planning lectures (Weeks 7–8), I created a custom domain file, `myTeam.pddl`.

It defines **types** (agent, enemy, team, position), **predicates** describing game facts, and **actions** for different strategies.

Some examples of predicates include:

- `(food_available)`
- `(territory_secure)`
- `(is_pacman ?a)`
- `(carrying_significant_food ?a)`
- `(being_chased ?a)`
- `(enemy_is_pacman ?e)`
- `(teammate_defending ?t)`

Together, these form a symbolic “snapshot” of the game that my planner reasons over.

4.2 Action Schemas

Each symbolic action has preconditions and effects, just like the STRIPS examples in Week 8.

Example – *Strategic Retreat*:

- (: action strategic-retreat
- :parameters (?a - current_agent)
- : precondition (and (is_pacman ?a)
- (carrying_significant_food ?a))
- : effect (and (at_home ?a)
- (not (is_pacman ?a))
- (not (carrying_significant_food ?a))))

This action fires when my Pacman is deep in enemy territory, loaded with food, and it's time to head home.

I added several new actions beyond the starter template, such as secure-power-capsule, cooperative-intercept, and aggressive-endgame, to cover more of the predicates available and to allow coordinated play.

4.3 Python Integration

In my myTeam.py, the PDDLPlanner converts the live game state into a predicate set, checks which PDDL actions are applicable, assigns a priority score, and returns the best one.

Rather than using an external PDDL solver (too slow per move), this works like “**heuristic planning as search**” from Week 9; it grounds the logic instantly and acts within 0.05s.

4.4 Explainability

Every chosen action can be explained through its satisfied predicates.

For example, when the logs show:

Choose a strategic retreat because the agent is Pac-Man, being chased, and carrying significant food.

That kind of symbolic reasoning links directly to Week 12 on **Explainable Planning**, giving my system a clear, human-readable justification for its moves.

5. Tactical Planning – Risk-Aware A*

For the low-level planner, I implemented a risk-weighted A* algorithm from **Weeks 2–4**.

The cost function is:

$$f(s) = g(s) + h(s) = (1 + \text{riskPenalty}(s)) + \text{mazeDistance}(s, \text{goal})$$

The risk penalty adds a big cost near visible ghosts and a smaller one near likely ghost positions from the particle filter.

Distance to ghost	Penalty	Effect
≤ 1	+1000	Avoid immediate death
≤ 2	+200	High caution
≤ 5	+10	Gentle avoidance

This approach made the pathfinder sensitive to danger while still efficient.

I also implemented **game-phase adaptation**, where thresholds change automatically between *early*, *mid*, and *late* game phases, as suggested in Week 9 on adaptive heuristic search.

6. Reasoning Under Uncertainty

Drawing from **Week 10 (MDPs and uncertainty)**, each opponent is tracked with a 250-particle filter.

Every turn:

1. Predict likely new positions.
2. Weight them by how well the noisy distance fits.
3. Resample and inject randomness if diversity drops.
4. Use the most likely position to estimate risk.

When the opponent becomes visible, all particles collapse to that exact tile.

This keeps beliefs accurate but computationally light enough for real-time play.

7. Cooperation and Multi-Agent Planning

Following **Week 5 (Multi-Agent Path Finding)**, I used a simple blackboard for coordination. Agents post:

- Claimed food targets (to avoid duplicates).
- Turn counts and phase info.
- Recent defensive food loss.

This shared memory helped them work together without expensive joint-state planning.

8. Role-Specific Strategies

Offensive Agent

Implements symbolic actions from the PDDL file:

- **Aggressive / Cautious attack** → pick unclaimed food by distance + risk cost.
- **Secure power capsule** → target nearest capsule.
- **Strategic retreat** → go to the safest home tile when chased or overloaded.

Defensive Agent

Focuses on protecting home territory:

1. Cluster defensive food positions (radius 5).
2. Choose the top 3 clusters by size.
3. Add high-visibility boundary entrances as patrol points.
4. Score each zone by distance, recency, and threat.
If an invader is detected, switch to interception mode for up to 10 turns, then resume patrol.

9. Experiments and Evaluation

Setup

- Maps: smallGrid, capture, custom maze.
- Opponents: baseline and strong defensive teams.
- 20 games per layout.
- Metrics: score, deaths per 100 turns, interceptions, computation time.

Results:

30 games vs staffTeam:

New vs Old Implementation — 30 matches each

Metric	Primary Impl	Secondary Impl	Δ (New–Old)
Games	30	30	0
Wins	26	30	–4
Losses	2	0	+2
Ties	2	0	+2
Win rate	0.87	1.00	–0.13
Avg margin	9.67	9.70	–0.03
Median margin	9	11	–2
Std dev (margin)	6.11	2.78	+3.33
Best margin	18	15	+3
Worst margin	–6	2	–8
Big wins (≥ 10)	13	17	–4
Close games (≤ 3 pts either way)	4	1	+3
Blowouts (≥ 15)	8	4	+4

Notes:

- Secondary impl is perfect on wins. Primary impl trades a few losses/ties for a higher ceiling (best margin +3) but more variance (std dev +3.33) and one negative outing (-6).
- Average margin is effectively the same. Median drops by 2 for the Primary impl, showing more spread.
- Primary impl produces more blowouts (≥ 15) but also more close games and two failures

Against staff baselines, the new PDDL+A*+PF agent averaged 21.08 points with a 97.96% win rate; against mixed ladder opponents, it averaged 28.64–16.91 (+11.73 margin) with a 63.64% win rate, a clear improvement over the pre-upgrade performance indicated by the old EP log.

10. Experimental Variations and What I Learned

Attempt	Purpose	Result	Limitation
Monte Carlo Tree Search	Escape planning	Worked conceptually	Too slow (> 0.8 s/turn)
Opponent Modelling	Predict aggression	Interesting	Unstable behaviour
Intent-Biased Filter	Smarter beliefs	Better tracking	CPU heavy
Map Analyzer	Identify safe zones	Helped debugging	Map-specific
TD-Lambda Learning	Learn weights	Converged slowly	Overfitted
Dynamic Patrols	Rotate defence	Good coverage	Expensive
Communication Protocol	Explicit messaging	Helped logging	Small impact

These experiments taught me how theory meets practice. Many of the “clever” ideas from lectures worked in principle but exceeded time budgets. Simplifying to a hybrid symbolic–heuristic model kept things practical and explainable.

11. Discussion

Strengths

- Integrates content from all stages: search, planning, uncertainty, and explainability.
- Decisions are transparent because of explicit PDDL reasoning.
- Cooperative behaviour without full joint planning.
- Fast enough for real-time contests.

Weaknesses

- Fixed action priorities (no adaptive meta-planner).
- Simplified motion model for hidden enemies.
- No global joint optimisation between teammates.

Insights

Working on this project made the weekly lectures feel connected from basic A* in Week 2 to explainable symbolic reasoning in Week 12. The CTF environment became a practical way to unify all those ideas.

12. Reflection and Future Work

Initially, I tried more data-driven and stochastic approaches (MCTS, TD-Lambda, learning weights). These related to **Weeks 10–11** on uncertainty and reinforcement learning, but they were too computationally expensive.

Switching to a PDDL-guided design gave me the best of both worlds: explicit logic from **Weeks 7–9** and efficient search from **Weeks 2–4**.

If I had more time, I'd experiment with:

- Automatically learning priority scores for PDDL actions.
- Adding probabilistic PDDL extensions.
- Partial joint planning so both agents coordinate deeper.

13. Conclusion

My final CTF system demonstrates how concepts from the whole FIT5222 syllabus come together in one working AI.

It uses search and heuristics for movement, PDDL for symbolic reasoning, particle filters for uncertainty, and blackboard memory for coordination.

By combining these elements, the agents make rational, explainable decisions in real time.

References

- Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- FIT5222 Lecture Slides, Weeks 1–12, Monash University.

Acknowledgement of Gen-AI

Portions of this report were developed with assistance from **Gen-AI** to improve clarity, formatting, and readability.

All technical design, coding, experimentation, and analytical content are my own original work.

ChatGPT was used solely for editorial support and summarising my implementation explanations in accordance with Monash University's academic integrity guidelines.

Gen-AI was used as a sparring partner while implementing multiple approaches for the Pac-Man agents.